

category: dotnet-general

tags: [rabbitmq, autorecovery, blocking-call, async-void, ui-thread, silent-hang]

severity: high

created: 2026-07-28

updated: 2026-07-28

project-origin: App-Access-V2 (iAccessDesktopv2.Avalonia)

RabbitMQ.Client BasicPublish block đồng bộ khi đang auto-recovery → treo luồng gọi trước khi tới code UI phía sau

Tình huống gặp phải

Màn "Giám sát" (Monitor) không hiển thị bất kỳ sự kiện quẹt thẻ nào dù cả 2 loại thiết bị (ZktecoPush qua HTTP push, và KZE02 KZTEK E-series) đều xác nhận có gửi dữ liệu lên app. Debug bằng breakpoint trong `MainViewModel.OnEventSendToServerAsync` (iAccessDesktopv2.Avalonia/ViewModels/Main/MainViewModel.cs).

Triệu chứng / Lỗi

Code:

```
if (!log.CardNo.Equals("0") && log.Pin.ToString().Equals("0"))
{
    App.EventBus?.Send(JsonConvert.SerializeObject(new {...}),
    RabbitMQConfig.ExchangeNameSentEventAccess);
}
// đoạn cập nhật RealtimeEvents.Insert(...) nằm NGAY SAU, không lồng trong if trên
```

User báo "treo khi kết nối RabbitMQ bị mất" — không có exception, không log lỗi, method đơn giản không chạy tiếp xuống được đoạn `RealtimeEvents.Insert(...)` (cập nhật lưới UI) nằm ngay phía dưới, dù đoạn đó nằm NGOÀI khối `if`.

Nguyên nhân gốc rễ (Root Cause)

`RabbitMqEventBus.Send()` chỉ kiểm tra `channel == null` trước khi gọi `channel.BasicPublish(...)`. Khi kết nối RabbitMQ bị mất nhưng `AutomaticRecoveryEnabled = true` (mặc định bật trong `Connect()`), `RabbitMQ.Client` bọc channel gốc bằng `AutorecoveringModel` — **object này không bao giờ null**, kể cả khi đang mất kết nối/đang cố tự kết nối lại. Guard `channel == null` do đó luôn pass.

Khi gọi `BasicPublish` trong lúc `AutorecoveringModel` đang trong quá trình

auto-recovery, RabbitMQ.Client **block đồng bộ** thread gọi cho tới khi recovery xong (hoặc treo vô hạn nếu broker không sống lại được, không có timeout mặc định rõ ràng cho case này). Vì `OnEventSendToServerAsync` gọi `Send()` một cách đồng bộ (không `await`, không `Task.Run`) ngay trước đoạn code cập nhật UI, khi `Send()` treo thì toàn bộ method treo theo — đoạn `RealtimeEvents.Insert(...)` phía sau không bao giờ chạy tới, dù về mặt code nó nằm ngoài khối `if`.
Bug này ảnh hưởng MỌI loại thiết bị (ZktecoPush, KZTEK E-series...) vì tất cả đều đi qua chung 1 method `OnEventSendToServerAsync`.

Giải pháp

1. `RabbitMqEventBus.Send()` — thêm guard `IsConnected` (property đã có sẵn: `conn?.IsOpen == true && channel?.IsOpen == true`) trước khi `BasicPublish`:

```
```csharp
if (channel == null || !IsConnected) return false;
`
```

2. Ở tầng gọi (`MainViewModel.OnEventSendToServerAsync`), đưa lời gọi `Send()` thành fire-and-forget (`_ = Task.Run(() => App.EventBus?.Send(...))`) — phòng hờ trường hợp race hiếm (kết nối OK lúc check `IsConnected` nhưng rơi vào recovery đúng lúc gọi `BasicPublish`), đảm bảo UI luôn cập nhật độc lập với trạng thái RabbitMQ.

## Áp dụng lại (How to reuse)

- Khi thấy **\*\*method đồng bộ** (không async, không throw) treo im lặng không rõ lý do\*\*, và trong method có gọi publish/send của RabbitMQ.Client (hoặc bất kỳ client nào có auto-recovery/reconnect nội bộ) → nghi ngờ ngay `BasicPublish`/tương đương đang block chờ recovery.
- Bất kỳ nơi nào gọi `RabbitMqEventBus.Send(...)` mà kết quả gọi đó có thể chặn đường code quan trọng phía sau (cập nhật UI, ghi DB, trả response) → **PHẢI** fire-and-forget hoặc đặt `Send` ở cuối cùng, sau khi các side-effect quan trọng đã chạy xong.
- Không tin `xxx == null` là đủ để biết "đối tượng còn dùng được" khi thư viện có auto-recovery/reconnect wrapper (object reference giữ nguyên xuyên suốt vòng đời reconnect) — phải kiểm state thật (`IsOpen/IsConnected`).

## Chú ý / Cạm bẫy (Gotchas)

- ⚠ Guard `IsConnected` không loại bỏ hoàn toàn race condition (kết nối rớt đúng giữa lúc `IsConnected` trả `true` và lúc `BasicPublish` chạy) — fire-and-forget ở tầng gọi là lớp phòng thủ thứ 2 bắt buộc phải có, không thể chỉ dựa vào guard

ở tầng `Send()`.

- ⚠ Vì không có exception/log khi treo kiểu này, log file (kể cả log HTML tự build của app) sẽ hoàn toàn im lặng — không thấy dòng nào ở điểm treo. Chỉ phát hiện được qua debugger breakpoint hoặc đọc code suy luận từ hành vi mô tả của user ("treo khi mất mạng"), không suy luận được chỉ từ log.

## Tham chiếu

- File: `iAccessDesktopv2.Avalonia/iAccess.Core/Messaging/RabbitMqEventBus.cs`  
(`Send()`, `Connect()` với `AutomaticRecoveryEnabled = true`)
- File:  
`iAccessDesktopv2.Avalonia/iAccessDesktopv2.Avalonia/ViewModels/Main/MainViewModel.cs`  
(`OnEventSendToServerAsync`)
- Project liên quan: App-Access-V2 (`iAccess/iAccessDesktopv2.Avalonia`)